Trabajo Práctico de Paradigmas de Programación

Integrantes:

- Santiago Cordoba
- Octavio Alejandro Gudiño
- Emanuel Salomón
- Renzo Scollo
- Franco Testi

Número de comisión: 205

Año lectivo: 2025

Entrega: Nro. 2

Grupo: Nro. 4

Enunciado de la situación problemática

Florería Arcoiris

La florería Arcoiris nos ha solicitado un sistema que le permita gestionar los envíos de ramos a domicilio que prepara, dentro y fuera de la ciudad de Rosario.

Cada ramo se prepara utilizando distintas variedades de flores. De cada variedad se deberá registrar: código, descripción y precio por unidad. Ejemplos de variedades de flores son: rosas, claveles, gladiolos, azucenas, violetas, etc.

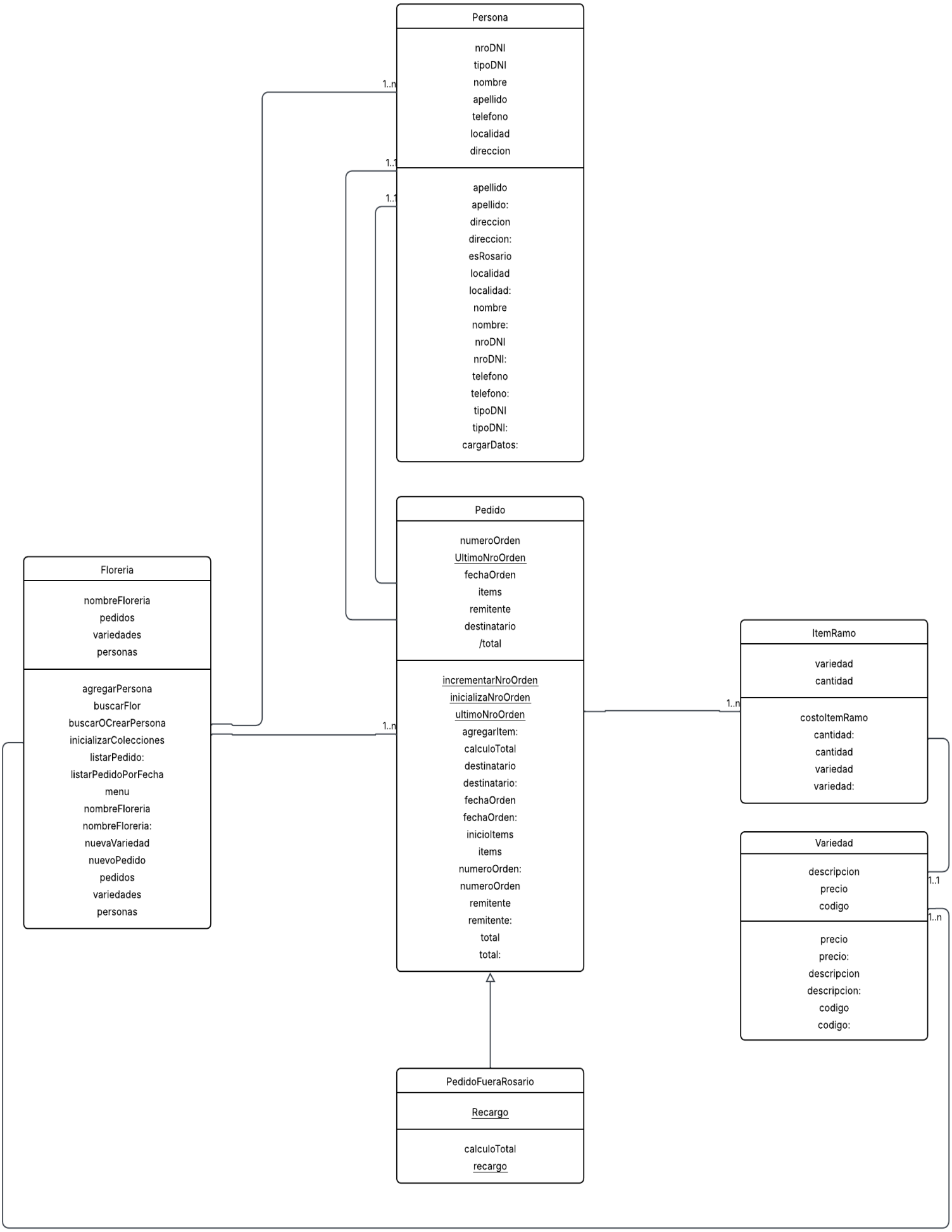
De cada pedido que se reciba se registra: número de orden (número correlativo asignado automáticamente por el sistema), fecha, remitente (persona que solicita el ramo), destinatario y composición del ramo (indicando variedad y cantidad de cada variedad de flor). Notar que un ramo puede tener diferentes variedades de flores y que un pedido contendrá un único ramo.

Tanto del remitente como del destinatario se registran los siguientes datos: tipo y número de documento, apellido, nombres, dirección, localidad y teléfono.

El costo del ramo se calculará en función del precio por unidad de cada variedad y de la cantidad de flores de dicha variedad que componen el ramo.

Si el pedido se envía fuera de la ciudad, al costo del ramo se le aplicará un porcentaje de recargo, que será el mismo para todos los pedidos a enviar fuera de Rosario.

La florería desea obtener un listado de los pedidos registrados en un período de fechas determinadas. Las fechas de inicio y fin del período se ingresarán por teclado. El listado deberá contener: fecha del pedido, descripción de la o las variedades que componen el ramo, localidad del destinatario si es un envío fuera de la ciudad de Rosario, costo del ramo y monto total de todos los envíos comprendidos en el listado.


Persona
nroDNI
tipoDNI
nombre
apellido
telefono
localidad
direccion
apellido
apellido:
direccion
direccion:
esRosario
localidad
localidad:
nombre
nombre:
nroDNI
nroDNI:
telefono
telefono:
tipoDNI
tipoDNI:
cargarDatos:
1..n
1..1
1..1
Pedido
numeroOrden
UltimoNroOrden
fechaOrden
items
remitente
destinatario
/total
incrementarNroOrden
inicializaNroOrden
ultimoNroOrden
agregarItem:
calculoTotal
destinatario
destinatario:
fechaOrden
fechaOrden:
inicioItems
items
numeroOrden:
numeroOrden
remitente
remitente:
total
total:
Floreria
nombreFloreria
pedidos
variedades
personas
agregarPersona
buscarFlor
buscarOCrearPersona
inicializarColecciones
listarPedido:
listarPedidoPorFecha
menu
nombreFloreria
nombreFloreria:
nuevaVariedad
nuevoPedido
pedidos
variedades
personas
1..n
1..n
ItemRamo
variedad
cantidad
costoItemRamo
cantidad:
cantidad
variedad
variedad:
1..n
Variedad
descripcion
precio
codigo
precio
precio:
descripcion
descripcion:
codigo
codigo:
1..1
1..n
PedidoFueraRosario
Recargo
calculoTotal
recargo

Programa principal utilizado desde el Workspace para ejecutar el sistema

```
flor := Floreria new.

flor menu.
```

Código Fuente en Smalltalk

https://drive.google.com/drive/folders/1DFRaRlvPxiOCWQNo8QFcBEvF5V9SM5fN?usp=drive_link

Cuestionario

a) Indique si encuentra clases abstractas. ¿Por qué son abstractas? Definir y dar ejemplos.

b) Indique si encuentra clases concretas. ¿Por qué son concretas? Definir y dar ejemplos.

c) ¿Qué relaciones de herencia reconoce? Dar ejemplos. Explicar el concepto de herencia.

d) ¿Qué relaciones de ensamble reconoce? Dar ejemplos. Explicar el concepto de ensamble.

e) ¿Existen métodos polimórficos? Dar ejemplos. Explicar el concepto de polimorfismo.

f) ¿Existen métodos con redefinición polimórfica? Dar ejemplos. Explicar el concepto de redefinición.

g) Identifique las variables de Clase utilizadas. Dar ejemplos. Explique qué son las variables de Clases y porque/cómo se utilizan en el presente trabajo.

h) Identifique las variables de Instancia utilizadas. Dar ejemplos. Explique qué son las variables de Instancia y sus diferencias con las variables de Clase.

i) Identifique los métodos de Clase. Dar ejemplos. ¿Por qué son necesarios los métodos de clase?

j) Identifique mensajes unarios, binarios y de palabra clave. De ejemplos de cada uno de ellos.

Nota: Todos los ejemplos solicitados en este cuestionario deben ser sacados del trabajo práctico que le haya tocado a cada grupo.

a) No se puede considerar a ninguna clase como abstracta ya que todas son instanciadas directamente durante la ejecución. Las clases abstractas no están diseñadas para ser instanciadas directamente; su propósito principal es servir como una base conceptual y funcional que otras clases pueden extender.

b) Se pueden considerar a las clases Persona,Pedido, Floreria, ItemRamo, PedidoFueraRosario y Variedad como concretas, ya que permiten instanciar objetos reales. Por ejemplo: una persona, un ramo. Una clase concreta es aquella de la cual se pueden crear objetos.

c) La herencia es el mecanismo por el cual una clase hereda atributos y métodos de otra. En este caso, PedidoFueraRosario hereda de Pedido, esto implica que PedidoFueraRosario tiene todos los atributos y métodos de Pedido.

d) Las relaciones de ensamble son relaciones entre objetos en las cuales un objeto está compuesto por otros objetos, que colaboran para cumplir con sus responsabilidades.

- Floreria ensambla a Pedido, Variedad y Persona.
- Pedido ensambla a ItemRamo y Persona.
- ItemRamo ensambla a Variedad.

e) El polimorfismo permite que distintas clases respondan al mismo mensaje con comportamientos diferentes, Pedido y PedidoFueraRosario comparten el metodo calculoTotal, este es un caso de polimorfismo, porque calculoTotal se comporta distinto según el tipo de pedido.

f) La redefinición polimórfica es la capacidad de una clase de modificar o redefinir un método que ya ha sido definido en su superclase. Esto permite a la subclase modificar o extender el comportamiento heredado de la superclase. Para que exista la redefinición es necesario que exista una relación de herencia.

Superclase: Pedido. Subclase: PedidoFueraRosario

El método calculoTotal en Pedido calcula el importe total del ramo.

calculoTotal es redefinido por PedidoFueraRosario para agregar un recargo por ser un envío fuera de la ciudad.

g) Las variables de clase pertenecen a la clase en sí misma y se comparten entre todas las instancias. En Pedido la variable UltimoNroOrden se usa junto a los métodos incrementarNroOrden, inicializarNroOrden y ultimoNroOrden, para controlar la numeración de pedidos. Sirven para almacenar información global que no depende de un pedido en particular, sino de todos los pedidos.

h) Las variables de instancia pertenecen a cada objeto individual y representan su estado.

En Persona: nombre, apellido, teléfono, localidad, dirección.

En Pedido: numeroOrden, fechaOrden, remitente, destinatario, items, total.

En Variedad: descripción, precio, código.

En ItemRamo: cantidad, variedad

En Floreria: nombreFlorería,pedidos,variedades,personas

Cada objeto tiene sus propias copias de las variables de instancia, mientras que las de clase son únicas y compartidas.

i) Métodos de clase: incrementarNroOrden, inicializaNroOrden, ultimoNroOrden,recargo:.

Los métodos de clase son necesarios para implementar la funcionalidad que es común o transversal a toda la clase o para administrar el estado compartido por todas sus instancias. Permiten gestionar variables de clase, que contienen información que debe ser única y consistente para todos los objetos de esa clase.

3/4

j)

Unario: mensaje sin argumentos

Binario: operador Unario: con un token corto

Palabra clave: mensaje con uno o más argumentos nombrados.

Unarios:

- Date today

- personas isNil
- variedades isEmpty
- self nombreFloreria

Binarios:

- UltimoNroOrden + 1
- ret := (ret + item costoItemRamo)
- variedad = nil
- (unPedido fechaOrden <= fin).

Palabra clave:

- Prompter prompt: '...'
- personas detect: [:x | x nroDNI = numero] ifNone: [nil]
- ped agregarItem: item
- ped numeroOrden: Pedido ultimoNroOrden
- variedades add: variedad

Código fuente, package:

```
| package |
package := Package name: 'Floreria 16-10-2025'.
package paxVersion: 1;
	basicComment: ''.

package classNames
	add: #Floreria;
	add: #ItemRamo;
	add: #Pedido;
	add: #PedidoFueraRosario;
	add: #Persona;
	add: #Variedad;
	yourself.

package binaryGlobalNames: (Set new
	yourself).

package globalAliases: (Set new
	yourself).

package setPrerequisites: #(
	'..\..\Documentos\Dolphin Smalltalk 7\Core\Object Arts\Dolphin\Base\Dolphin'
	'..\..\Documentos\Dolphin Smalltalk 7\Core\Object Arts\Dolphin\Base\Dolphin Legacy Date & Time'
	'..\..\Documentos\Dolphin Smalltalk 7\Core\Object Arts\Dolphin\Base\Dolphin Message Box'
```

```
	'..\..\Documentos\Dolphin Smalltalk 7\Core\Object Arts\Dolphin\MVP\Presenters\Prompters\Dolphin Prompter').

package!

"Class Definitions"!

Object subclass: #Floreria
	instanceVariableNames: 'nombreFloreria variedades pedidos personas'
	classVariableNames: ''
	poolDictionaries: ''
	classInstanceVariableNames: ''!

Object subclass: #ItemRamo
	instanceVariableNames: 'variedad cantidad'
	classVariableNames: ''
	poolDictionaries: ''
	classInstanceVariableNames: ''!

Object subclass: #Pedido
	instanceVariableNames: 'numeroOrden fechaOrden remitente destinatario items total'
	classVariableNames: 'UltimoNroOrden'
	poolDictionaries: ''
	classInstanceVariableNames: ''!

Object subclass: #Persona
```

```
	instanceVariableNames: 'nroDNI tipoDNI nombre apellido telefono direccion localidad'
	classVariableNames: ''
	poolDictionaries: ''
	classInstanceVariableNames: ''!

Object subclass: #Variedad
	instanceVariableNames: 'codigo descripcion precio'
	classVariableNames: ''
	poolDictionaries: ''
	classInstanceVariableNames: ''!

Pedido subclass: #PedidoFueraRosario
	instanceVariableNames: ''
	classVariableNames: 'Recargo'
	poolDictionaries: ''
	classInstanceVariableNames: ''!

"End of package definition"!

"Source Globals"!

"Classes"!

Floreria guid: (GUID fromString: '{553295d0-7e8c-44c1-92c5-713f094c0959}')!

Floreria comment: ''!
```

```
!Floreria categoriesForClass!Kernel-Objects! !


!Floreria methodsFor!


agregarPersona
|per|
per := self buscarOCrearPersona.
personas add: per.
^per
!


buscarFlor
|variedad cod|
   variedad := nil.
    [variedad = nil] whileTrue: [
        cod := (Prompter prompt: 'Ingrese código de flor (-
1 para terminar)') asNumber.
     variedad:= (variedades detect: [:v | v codigo = cod]
ifNone: [nil]).
     (variedad = nil) ifTrue:[MessageBox notify:'No existe
la variedad con dicho codigo'].
          ].
^variedad


!


buscarOCrearPersona
| numero pers |
```

```
numero := (Prompter prompt: 'Ingrese su DNI: ').

pers := personas detect: [:x | x nroDNI = numero] ifNone:
[nil].

pers isNil ifTrue: [
    pers := Persona new.
    pers cargarDatos: numero.
].

^pers
!

cargaDatosPrueba
|ped item pers remitente destinatario var|
var := Variedad new.
var descripcion: 'Rosa'.
var codigo: 1.
var precio: 100 asFloat.
variedades add: var.
var := Variedad new.
var descripcion: 'Margarita'.
var codigo: 2.
var precio: 200 asFloat.
variedades add: var.
var := Variedad new.
var descripcion: 'Petuña'.
var codigo: 3.
```

```
var precio: 300 asFloat .
variedades add: var.
"PEDIDO 1"
pers := Persona new.
pers nombre: 'Juan'.
pers apellido: 'Johnson'.
pers localidad: 'Rosario'.
pers nroDNI: '46366884'.
pers tipoDNI: 'A'.
pers direccion: 'Av. Pellegrini 805'.
pers telefono: '3415623980'.
personas add: pers.
remitente := pers.
pers := Persona new.
pers nombre: 'Pablo'.
pers apellido: 'Ferrari'.
pers localidad: 'Rosario'.
pers nroDNI: '47456543'.
pers tipoDNI: 'A'.
pers direccion: 'Vespucio 4956'.
pers telefono: '3415692432'.
personas add: pers.
destinatario :=  pers.
(destinatario esRosario) ifTrue: [ped := Pedido new]
ifFalse: [ped := PedidoFueraRosario new].
ped inicioItems.
Pedido incrementaUltimoNroOrden.
ped numeroOrden: Pedido ultimoNroOrden.
```

```
ped destinatario: destinatario.
item := ItemRamo new.
item variedad: (variedades at:1).
item cantidad: 5.
ped agregarItem: item.
item := ItemRamo new.
item variedad: (variedades at:2).
item cantidad: 3.
ped agregarItem: item.
ped destinatario: destinatario.
ped remitente: remitente.
ped total: ped calculoTotal.
ped fechaOrden: Date yesterday.
pedidos add: ped.
"PEDIDO 2"
remitente := (personas at:1).
pers := Persona new.
pers nombre: 'Mateo'.
pers apellido: 'Martín'.
pers localidad: 'Peyrano'.
pers nroDNI: '44543982'.
pers tipoDNI: 'A'.
pers direccion: 'Bv. Lavalle 13'.
pers telefono: '3402564391'.
personas add: pers.
destinatario :=  pers.
(destinatario esRosario) ifTrue: [ped := Pedido new]
ifFalse: [ped := PedidoFueraRosario new].
```

```
ped inicioItems.
Pedido incrementaUltimoNroOrden.
ped numeroOrden: Pedido ultimoNroOrden.
ped destinatario: destinatario.
item := ItemRamo new.
item variedad: (variedades at:3).
item cantidad: 4.
ped agregarItem: item.
item := ItemRamo new.
item variedad: (variedades at:1).
item cantidad: 6.
ped agregarItem: item.
ped destinatario: destinatario.
ped remitente: remitente.
ped total: ped calculoTotal.
ped fechaOrden: Date today.
pedidos add: ped.!

inicializarColecciones
     variedades := OrderedCollection new.
     pedidos := OrderedCollection new.
     personas := OrderedCollection new.
!

inicializarRecargo
     PedidoFueraRosario Recargo: 5.!

listarPedido: unPedido
```

```
    | factura pedidoTemporal |
factura := String new.
    1 to: unPedido items size do: [:i |
        pedidoTemporal := unPedido items at: i.
        factura := factura,
            i printString , String tab ,
            (pedidoTemporal variedad codigo) printString ,
String tab ,
            (pedidoTemporal variedad descripcion) , String
tab , String tab,
            (pedidoTemporal cantidad) printString , String
tab ,
            (pedidoTemporal costoItemRamo) printString ,
String cr
    ].
Transcript
     show: self nombreFloreria asUppercase; cr;
     show: '-----------------------------------------------
-------------'; cr;
     show: 'Pedido número: ', unPedido numeroOrden
printString; cr;
     show: 'Fecha: ', unPedido fechaOrden printString; cr;
     show: 'Remitente:';
     show: '  ', unPedido remitente nombre, ' ', unPedido
remitente apellido; cr;
     show: 'Dirección: ', unPedido remitente direccion; cr;
cr;
     show: 'Destinatario:';
     show: '  ', unPedido destinatario nombre, ' ',
unPedido destinatario apellido; cr;
```

```
     show: 'Dirección: ', unPedido destinatario direccion;
cr; cr;

     show: 'Detalle del pedido:'; cr;

     show: 'Nº', String tab, 'Código', String tab,
'Descripción', String tab, 'Cantidad', String tab,
'Subtotal'; cr;

     show: '-----------------------------------------------
-------------'; cr;

     show: factura; cr;

     show: '-----------------------------------------------
-------------'; cr;

     show: 'TOTAL: ', unPedido total printString; cr;

     show: 'Gracias por confiar en ', self nombreFloreria;
cr;cr;cr.




!


listarPedidoPorFecha

|inicio fin pedidosSeleccionados|

inicio := Date fromString: (Prompter prompt:'Ingresar fecha
de inicio de periodo').

fin := Date fromString: (Prompter prompt:'Ingresar fecha de
fin de periodo' ).


pedidosSeleccionados := pedidos select: [:unPedido |

    (unPedido fechaOrden <= fin) and: [unPedido fechaOrden
>= inicio]].


     (pedidosSeleccionados isEmpty)
```

```
     ifTrue:[MessageBox notify:'No hay pedidos en el rango
solicitado'.]

     ifFalse:[

     Transcript

          show: 'Fecha de inicio del reporte: ', inicio
printString; cr;

          show: 'Fecha de finalización del reporte: ', fin
printString;cr.

     pedidosSeleccionados do:[:unPedido | self
listarPedido:unPedido].

].!


menu

|opc|

opc := 3.

Pedido inicializaUltimoNroOrden.

self inicializarColecciones.

self cargaDatosPrueba.

self inicializarRecargo.

[opc = 0] whileFalse: [

MessageBox notify: 'MENU

1- Nuevo Pedido

2- Reporte Pedidos

3- Crear nueva variedad

0- Salir'.

opc := (Prompter prompt: 'Ingrese un número entre 0 y 3')
asNumber.

(opc = 1) ifTrue:[self nuevoPedido].

(opc = 2) ifTrue:[self listarPedidoPorFecha].
```

```
(opc = 3) ifTrue:[self nuevaVariedad].
(opc = 0) ifTrue:[].
].!

nombreFloreria
     nombreFloreria := 'Arcoíris'.
     ^nombreFloreria!

nombreFloreria: anObject
     nombreFloreria := anObject!

nuevaVariedad
|variedad opc|
opc := 'S'.
[opc = 'S'] whileTrue:[
variedad := Variedad new.
variedad codigo: (Prompter prompt: 'Ingrese un codigo para
la flor que creará') asNumber.
variedad descripcion: (Prompter prompt: 'Ingrese una
descripción para esta flor').
variedad precio: (Prompter prompt: 'Ingrese el precio de
esta variedad.') asNumber asFloat.
variedades add: variedad.
opc := Prompter prompt: 'Ingrese S para continuar, de lo
contrario dejará de añadir variedades y será devuelto al
menú principal.'.].
!

nuevoPedido
```

```
| ped opc remitente destinatario sigue item|

sigue := 'S'.
opc := 'S'.
[opc = 'S'] whileTrue: [
    variedades isEmpty ifTrue: [
        MessageBox notify: 'No hay variedades cargadas'.
        opc := 'N'.
        ^self
    ].
    MessageBox notify: 'Ingrese datos del remitente'.
    remitente := self agregarPersona.
    MessageBox notify: 'Ingrese datos del destinatario'.
    destinatario := self agregarPersona.
    (destinatario esRosario) ifTrue: [ped := Pedido new]
     ifFalse: [ped := PedidoFueraRosario new].
    ped inicioItems.
    Pedido incrementaUltimoNroOrden.
    ped numeroOrden: Pedido ultimoNroOrden.
    ped remitente: remitente.
    ped destinatario: destinatario.

     [sigue = 'S'] whileTrue:[
     item := ItemRamo new.
     item variedad: self buscarFlor.
     item cantidad: (Prompter prompt: 'Ingrese cantidad')
asNumber.
```

```
    ped agregarItem: item.
     sigue := Prompter prompt:'¿Quiere Ingresar otro item? Para seguir ingrese S.'].

    ped total: ped calculoTotal.
    ped fechaOrden: Date today.
    pedidos add: ped.

    opc := Prompter prompt: 'Digite S para ingresar otro pedido, de lo contrario será enviado al menú principal'.
].
!

pedidos
     ^pedidos!

pedidos: anObject
     pedidos := anObject!

solicitarPedido "Cambiar clave principal por fecha"
|ret|
ret := (Prompter prompt: 'Ingrese el numero de pedido que quiere mostrar en pantalla')asNumber.
self listarPedido:(self pedidos at:(ret))
!

variedades
     ^variedades!
```

```
variedades: anObject
     variedades := anObject! !

!Floreria categoriesForMethods!
agregarPersona!public! !
buscarFlor!public! !
buscarOCrearPersona!public! !
cargaDatosPrueba!public! !
inicializarColecciones!public! !
inicializarRecargo!public! !
listarPedido:!public! !
listarPedidoPorFecha!public! !
menu!public! !
nombreFloreria!accessing!private! !
nombreFloreria:!accessing!private! !
nuevaVariedad!public! !
nuevoPedido!public! !
pedidos!accessing!private! !
pedidos:!accessing!private! !
solicitarPedido!public! !
variedades!accessing!private! !
variedades:!accessing!private! !
!

ItemRamo guid: (GUID fromString: '{e9c465bb-c9e9-4d82-9149-
062d5abf0edd}')!

ItemRamo comment: ''!
```

```
!ItemRamo categoriesForClass!Kernel-Objects! !

!ItemRamo methodsFor!

cantidad
	^cantidad!

cantidad: anObject
	cantidad := anObject!

costoItemRamo
^((self variedad precio) * self cantidad).!

variedad
	^variedad!

variedad: anObject
	variedad := anObject! !

!ItemRamo categoriesForMethods!
cantidad!accessing!private! !
cantidad:!accessing!private! !
costoItemRamo!public! !
variedad!accessing!private! !
variedad:!accessing!private! !
!
```

```
Pedido guid: (GUID fromString: '{9b911237-5bdd-429b-ba50-
faebc9b10811}')!

Pedido comment: ''!

!Pedido categoriesForClass!Kernel-Objects! !

!Pedido methodsFor!

agregarItem: unItem
    items add: unItem.!

calculoTotal
|ret|
ret := 0.
(self items) do:[:item| ret := (ret + item costoItemRamo)].
^ret.!

destinatario
     ^destinatario!

destinatario: anObject
     destinatario := anObject!

fechaOrden
     ^fechaOrden!

fechaOrden: anObject
```

```
	fechaOrden := anObject!

inicioItems
	items := OrderedCollection new.!

items
	^items!

items: anObject
	items := anObject!

numeroOrden
	^numeroOrden!

numeroOrden: anObject
	numeroOrden := anObject!

remitente
	^remitente!

remitente: anObject
	remitente := anObject!

total
	^total!

total: anObject
	total := anObject! !
```

```
!Pedido categoriesForMethods!
agregarItem:!public! !
calculoTotal!public! !
destinatario!accessing!private! !
destinatario:!accessing!private! !
fechaOrden!accessing!private! !
fechaOrden:!accessing!private! !
inicioItems!public! !
items!accessing!private! !
items:!accessing!private! !
numeroOrden!accessing!private! !
numeroOrden:!accessing!private! !
remitente!accessing!private! !
remitente:!accessing!private! !
total!accessing!private! !
total:!accessing!private! !
!

!Pedido class methodsFor!

incrementaUltimoNroOrden
UltimoNroOrden := UltimoNroOrden + 1!

inicializaUltimoNroOrden
UltimoNroOrden := 0!

ultimoNroOrden
```

```
^UltimoNroOrden! !

!Pedido class categoriesForMethods!
incrementaUltimoNroOrden!public! !
inicializaUltimoNroOrden!public! !
ultimoNroOrden!public! !
!

Persona guid: (GUID fromString: '{0879b65e-4079-4ca4-b71d-
d84d3368a70e}')!

Persona comment: ''!

!Persona categoriesForClass!Kernel-Objects! !

!Persona methodsFor!

apellido
	^apellido!

apellido: anObject
	apellido := anObject!

cargarDatos: unNumero"Parametro del metodo"
nroDNI:= unNumero.
tipoDNI:= (Prompter prompt: 'Ingrese el tipo del DNI:').
nombre:= (Prompter prompt: 'Ingrese el nombre de la persona
a cargar:').
```

```
apellido:= (Prompter prompt: 'Ingrese el apellido de la
persona a cargar:').

telefono:= (Prompter prompt: 'Ingrese el teléfono de la
persona, (sea destinatario o remitente), para
comunicarnos:').

localidad:= (Prompter prompt: 'Ingrese la localidad de la
persona:').

direccion:= (Prompter prompt: 'Ingrese la dirección de la
persona:').!


direccion

     ^direccion!


direccion: anObject

     direccion := anObject!


esRosario

(self localidad = 'Rosario')

     ifTrue:[^true]

     ifFalse:[^false].!


localidad

     ^localidad!


localidad: anObject

     localidad := anObject!


nombre

     ^nombre!
```

```
nombre: anObject
	nombre := anObject!

nroDNI
	^nroDNI!

nroDNI: anObject
	nroDNI := anObject!

telefono
	^telefono!

telefono: anObject
	telefono := anObject!

tipoDNI
	^tipoDNI!

tipoDNI: anObject
	tipoDNI := anObject! !

!Persona categoriesForMethods!
apellido!accessing!private! !
apellido:!accessing!private! !
cargarDatos:!public! !
direccion!accessing!private! !
direccion:!accessing!private! !
```

```
esRosario!public! !
localidad!accessing!private! !
localidad:!accessing!private! !
nombre!accessing!private! !
nombre:!accessing!private! !
nroDNI!accessing!private! !
nroDNI:!accessing!private! !
telefono!accessing!private! !
telefono:!accessing!private! !
tipoDNI!accessing!private! !
tipoDNI:!accessing!private! !
!

Variedad guid: (GUID fromString: '{c9a53c8d-d9c1-467b-a189-
9a20d1618011}')!

Variedad comment: ''!

!Variedad categoriesForClass!Kernel-Objects! !

!Variedad methodsFor!

codigo
	^codigo!

codigo: anObject
	codigo := anObject!
```

```
descripcion
	^descripcion!

descripcion: anObject
	descripcion := anObject!

precio
	^precio!

precio: anObject
	precio := anObject! !

!Variedad categoriesForMethods!
codigo!accessing!private! !
codigo:!accessing!private! !
descripcion!accessing!private! !
descripcion:!accessing!private! !
precio!accessing!private! !
precio:!accessing!private! !
!

PedidoFueraRosario guid: (GUID fromString: '{f5df7904-be5f-4461-af06-84cbd51225f9}')!

PedidoFueraRosario comment: ''!

!PedidoFueraRosario categoriesForClass!Kernel-Objects! !
```

```
!PedidoFueraRosario methodsFor!

calculoTotal
|ret|
ret := super calculoTotal.
ret := ret + (ret*(PedidoFueraRosario recargo) / 100).
^ret.! !

!PedidoFueraRosario categoriesForMethods!
calculoTotal!public! !
!

!PedidoFueraRosario class methodsFor!

recargo
     ^Recargo!

Recargo: anObject
     Recargo := anObject.! !

!PedidoFueraRosario class categoriesForMethods!
recargo!public! !
Recargo:!public! !
!

"Binary Globals"!
```